

Project 1 Report

INFSCI 2591 - Algorithm Design

Abhay Kumara Sri Krishna Nandiraju
University of Pittsburgh

Spring 2026

1 Merging Sorted Subarrays

Design and implement your own algorithm that takes the array A with size $m+n$ as input where: Subarray A[1], A[2],...A[m] sorted in ascending order and Subarray A[m+1], A[m+2],...A[n] sorted in ascending order and merges the two subarrays using an auxiliary array Aux of size $\min\{m,n\}$ back into array A sorted in ascending order. You must design and implement your own sorting function. Use of sorting functions in libraries is not permitted.

Test Case 1: {} and {3, 7, 9}

Test Case 2: {2, 7, 9} and {1}

Test Case 3: {1, 7, 10, 15} and {3, 8, 12, 18}

Test Case 4: {1, 3, 5, 5, 15, 18, 21} and {5, 5, 6, 8, 10, 12, 16, 17, 17, 20, 25, 28}

Code file `problem1.py` contains the implementation of the above problem. The results from running the code gives sorted array A as follows:

```
python problem1.py
--- Problem 1 Results ---
```

Test Case 1:

Original: [3, 7, 9]

Merged: [3, 7, 9]

Test Case 2:

Original: [2, 7, 9, 1]

Merged: [1, 2, 7, 9]

Test Case 3:

Original: [1, 7, 10, 15, 3, 8, 12, 18]

Merged: [1, 3, 7, 8, 10, 12, 15, 18]

Test Case 4:

Original: [1, 3, 5, 5, 15, 18, 21, 5, 5, 6, 8, 10, 12, 16, 17, 17, 20, 25, 28]

Merged: [1, 3, 5, 5, 5, 5, 6, 8, 10, 12, 15, 16, 17, 17, 18, 20, 21, 25, 28]

2 Multiplication

Design and implement your own algorithms, one for Ala Carte Multiplication (to break down the multiplication process into smaller, more manageable parts) and one for Rectangle Multiplication (to create a grid (or rectangle) where the area represents the product of the two numbers). Your algorithms must allow for both positive and negative multiplicands and multipliers.

```
Test Case 1: 7000 * 7294
Test Case 2: 25 * 5038385
Test Case 3: -59724 * 783
Test Case 4: 8516 * -82147953548159344
Test Case 5: 45952456856498465985 * 98654651986546519856
Test Case 6: -45952456856498465985 * -98654651986546519856
```

2.1 AlaCarte Multiplication

Code file `problem2_alacarte.py` contains the implementation of Ala Carte Multiplication. The results from running the code gives the products as follows:

```
python problem2_alacarte.py
--- Problem 2 Alacarte Results ---
Test Case 1:
  Num1: 7000
  Num2: 7294
  Product: 51058000

Test Case 2:
  Num1: 25
  Num2: 5038385
  Product: 125959625

Test Case 3:
  Num1: -59724
  Num2: 783
  Product: -46763892

Test Case 4:
  Num1: 8516
  Num2: -82147953548159344
  Product: -699571972416124973504

Test Case 5:
  Num1: 45952456856498465985
  Num2: 98654651986546519856
  Product: 4533423639104649634397093450504343098160

Test Case 6:
  Num1: -45952456856498465985
  Num2: -98654651986546519856
  Product: 4533423639104649634397093450504343098160
```

2.2 Rectangle Multiplication

Code file `problem2_rectangle.py` contains the implementation of Rectangle Multiplication. The results from running the code gives the products as follows:

```
python problem2_rectangle.py
--- Problem 2 Rectangle Results ---
Test Case 1:
  Num1: 7000
  Num2: 7294
  Product: 51058000

Test Case 2:
  Num1: 25
  Num2: 5038385
  Product: 125959625

Test Case 3:
  Num1: -59724
  Num2: 783
  Product: -46763892

Test Case 4:
  Num1: 8516
  Num2: -82147953548159344
  Product: -699571972416124973504

Test Case 5:
  Num1: 45952456856498465985
  Num2: 98654651986546519856
  Product: 4533423639104649634397093450504343098160

Test Case 6:
  Num1: -45952456856498465985
  Num2: -98654651986546519856
  Product: 4533423639104649634397093450504343098160
```

Note: In Python, the built-in integer type can handle arbitrarily large integers, so we can directly compute the products without worrying about overflow. If the algorithms are implemented in C++ or Java, we might need to handle large integers during computation.